

Dokumentation zur Einrichtung der lokalen Entwicklungsumgebung

Diese Dokumentation dient als Leitfaden für das Onboarding neuer Entwickler und beschreibt die initiale Einrichtung der Systemkomponenten sowie der isolierten Laufzeitumgebung für das Projekt.

1. Systemvoraussetzungen

Vor der Konfiguration des Projekts müssen die folgenden Kernkomponenten auf dem Zielsystem installiert werden.

1.1 Git (Versionsverwaltung)

- **Download:** Laden Sie die passende Git-Version für Ihr Betriebssystem von der offiziellen Git-Webseite herunter.
- **Installation:** Den Installationsassistenten mit den Standardeinstellungen ausführen.

1.2 Laufzeitumgebung (Python)

- **Download:** Laden Sie die aktuelle Python-Version von der offiziellen Python-Webseite herunter.
- **Wichtiger Installationshinweis (Windows):** Im initialen Installationsfenster muss die Option **"Add python.exe to PATH"** zwingend aktiviert werden. Dies stellt sicher, dass der Python-Interpreter global über die System-Shell aufgerufen werden kann.

2. Globale Git-Konfiguration

Nach erfolgreicher Installation von Git muss die Identität für die Erstellung von Commits in der System-Shell (Terminal) hinterlegt werden. Öffnen Sie hierzu ein Terminal in Visual Studio Code und führen Sie folgende Befehle aus:

```
git config --global user.name "Ihr GitHub-Benutzername"
git config --global user.email "Ihre GitHub-E-Mail-Adresse"
```

3. Repository klonen

Um eine lokale Arbeitskopie des Quellcodes zu erstellen, gehen Sie wie folgt vor:

1. Öffnen Sie Visual Studio Code.
2. Navigieren Sie über die Seitenleiste zur **Quellcodeverwaltung** (Shortcut: Strg + Shift + G).
3. Wählen Sie **Repository klonen -> Von GitHub klonen**.
4. Autorisieren Sie den Zugriff über den Browser und wählen Sie das gemeinsame Projekt-Repository aus.
5. Bestimmen Sie das lokale Zielverzeichnis auf Ihrer Festplatte.

6. Bestätigen Sie den nachfolgenden Dialog mit **Ordner öffnen**.

4. Einrichtung der virtuellen Laufzeitumgebung (Venv)

Zur Isolation von Projektabhängigkeiten und zur Vermeidung von Versionskonflikten wird eine virtuelle Umgebung verwendet. Visual Studio Code automatisiert diesen Prozess.

4.1 Automatisches Setup über VS Code

1. Stellen Sie sicher, dass die offizielle **Python-Erweiterung (Microsoft)** in VS Code installiert ist.
2. Öffnen Sie die Befehlspalette über **Strg + Shift + P**.
3. Suchen und wählen Sie den Befehl: **Python: Create Environment** (Umgebung erstellen).
4. Wählen Sie den Typ **Venv** und anschließend den installierten Python-Interpreter aus.
5. Die Erweiterung erkennt die im Repository hinterlegte `requirements.txt` automatisch. Aktivieren Sie das Kontrollkästchen für diese Datei, um alle benötigten Abhängigkeiten (inkl. FastAPI und Uvicorn) im Hintergrund zu installieren.

4.2 PowerShell-Berechtigungen anpassen (Windows-spezifisch)

Windows-Systeme blockieren standardmäßig die Ausführung von Skripten innerhalb der PowerShell. Um die Aktivierung der virtuellen Umgebung zu ermöglichen, muss die Ausführungsrichtlinie einmalig angepasst werden:

1. Öffnen Sie ein neues Terminal in VS Code.
2. Führen Sie folgenden Befehl aus:

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

3. Bestätigen Sie die Sicherheitsabfrage mit **J** (Ja) bzw. **Y** (Yes).

5. Validierung der Umgebung und Serverstart

Nach Abschluss der Konfiguration kann die Funktionalität des Backends lokal überprüft werden.

1. Schließen Sie das aktuelle Terminal und öffnen Sie ein neues Terminal-Fenster in VS Code, um die aktualisierten Umgebungsvariablen zu laden.
2. Das Terminal sollte nun automatisch mit dem Präfix **(.venv)** starten. Dies signalisiert die aktive, isolierte Umgebung.
3. Starten Sie den lokalen Entwicklungsserver über folgenden Befehl:

```
uvicorn main:app --reload
```

4. Der Server ist nun unter der lokalen Standard-Hostadresse erreichbar. Die interaktive API-Dokumentation (Swagger UI) kann über den entsprechenden Unterpfad `docs` aufgerufen werden.

6. Deployment- und Collaboration-Richtlinien

Um einen sauberen Workflow innerhalb des Repositories zu gewährleisten, sind folgende Best Practices einzuhalten:

- **Aktualität sichern:** Vor Beginn einer neuen Arbeitssitzung ist zwingend ein `git pull` (Änderungen synchronisieren) durchzuführen, um den lokalen Codebasis-Stand mit dem

Remote-Repository abzugleichen.

- **Isolierte Commits:** Änderungen sollten in logische, in sich geschlossene Einheiten unterteilt und mit einer präzisen Commit-Nachricht versehen werden (z. B. feat: Add authentication endpoint).
- **Abhängigkeiten dokumentieren:** Sollten neue Pakete über pip installiert werden, muss die requirements.txt vor dem Commit mithilfe von pip freeze > requirements.txt aktualisiert werden.